

CSCE 647 IMAGE SYNTHESIS · MODULE 04 · LECTURE NOTES

Acceleration Structures

Turning $O(n)$ per ray into $O(\log n)$, and the heuristic that does it.

MODULE OUTCOMES

- Explain the trade-off between spatial and object subdivision.
- Build a BVH with the surface area heuristic and justify every term.
- Implement efficient traversal with correct ordering and early exit.
- Explain why the SAH works, and what assumptions it makes.
- Measure and interpret traversal statistics.

1 The arithmetic that forces the issue

Consider a 1920×1080 image at 256 samples per pixel with a maximum path depth of 8. That is roughly 4×10^{10} rays, and next-event estimation (Module 08) will roughly double it.

A modest production scene has 10^6 triangles. Testing every triangle against every ray is 4×10^{16} intersection tests. At an optimistic 10^9 tests per second, that is about **1.3 million seconds per core** — call it 130,000 years on one core, or a few centuries on a large machine.

With a well-built BVH, a ray visits on the order of 100 nodes and tests perhaps 20 triangles. The same image becomes hours. **This is not an optimisation; ray tracing does not exist without it.**

2 Two families

	Spatial subdivision	Object subdivision
Examples	kd-tree, uniform grid, octree, BSP.	Bounding volume hierarchy (BVH).
Partitions	Space , into disjoint regions.	Primitives , into disjoint groups, each with a bounding box.
Primitive placement	A primitive straddling a boundary appears in several regions.	Each primitive is in exactly one leaf.
Region overlap	None . Regions are disjoint by construction.	Bounds may overlap , so a ray may have to descend into both children.

	Spatial subdivision	Object subdivision
Traversal consequence	Can process regions in strict front-to-back order and stop at the first hit found.	Must consider both children when their bounds overlap, though near-first ordering recovers most of the benefit.
Memory	Unbounded in n — depends on how badly primitives straddle splits.	At most $2n - 1$ nodes. Predictable.
Build time	Minutes for large scenes.	Seconds.
Animation	Rebuild from scratch.	Refit bounds without changing topology.

WHY THE BVH WON

For about fifteen years the kd-tree was understood to be faster for static scenes, and for static scenes it often still is.

It lost on everything else. Memory consumption cannot be predicted from primitive count, which is intolerable when a scene must fit in a fixed budget. Build times of minutes are unacceptable when geometry deforms every frame. And there is no way to refit a kd-tree for animation — the splitting planes are tied to the geometry's positions.

The BVH's bounded $2n - 1$ node count, second-scale builds, and refittability matched what production actually needed.

Then hardware settled it. Every GPU with ray tracing support implements BVH traversal in fixed-function silicon. The question is no longer open.

3 The surface area heuristic

Given a set of primitives, which split is best? The surface area heuristic answers this by estimating the expected cost of each candidate, and it rests on one result from integral geometry.

$$P(\text{ray hits B} \mid \text{ray hits A}) = SA(B) / SA(A)$$

For convex B contained in convex A, and rays drawn from a uniform distribution over directions and origins, the conditional probability of hitting the inner box given that you hit the outer one is the ratio of their **surface areas**.

SURFACE AREA, NOT VOLUME

The intuition most people reach for is volume, and it is wrong. A ray is a line, not a point: it enters a box through its *surface*, and the measure of lines intersecting a convex body is proportional to that body's surface area.

Consider a very flat box — a slab. Its volume is nearly zero, but it has substantial surface area and a great many rays pass through it. Any volume-based heuristic would consider it free, and it is not.

This is why the SAH is the surface *area* heuristic, and getting that right is most of why it works as well as it does.

$$\text{cost} = C_{\text{trav}} + (SA(L) \cdot N(L) + SA(R) \cdot N(R)) \cdot C_{\text{isect}} / SA(P)$$

Term	Meaning
C_{trav}	Cost of visiting an interior node and testing two child boxes. Measured, not guessed — instrument your renderer.
C_{isect}	Cost of one primitive intersection test.
$SA(L)/SA(P)$	Probability a ray entering the parent also enters the left child.
$N(L)$	Number of primitives in the left child.

THE SAH MINIMISES EXPECTED COST, NOT TREE BALANCE

A median split divides primitives evenly and produces a balanced tree. That optimises the wrong thing.

The SAH optimises **expected traversal work**, and the two objectives frequently disagree. The SAH will happily put 90% of the primitives on one side if doing so puts a tightly clustered group behind a *small* bounding box that few rays enter.

And that is correct. A node that rays rarely visit contributes almost nothing to the expected cost, no matter how many primitives it contains. Balance is irrelevant; expected work is what you pay.

In practice the SAH beats median splitting by a factor of two to three in traversal cost. That is the difference between a renderer you can iterate with and one you cannot.

3.1 Binning

Evaluating every possible split exactly requires sorting the primitives along each axis, giving an $O(n \log^2 n)$ build. **Binned SAH** instead divides the centroid bounds into a fixed number of bins — 12 to 16 is standard — accumulates each primitive into one bin in a single pass, and then evaluates only the bin boundaries as candidate splits, using prefix and suffix sweeps to make each candidate's cost $O(1)$.

This gives roughly 99% of the quality of the exact SAH at a small fraction of the build cost, and it is what essentially every production builder does.

THE TERMINATION TEST IS NOT OPTIONAL

```
if (best_cost >= N * C_isect) return make_leaf(prims);
```

This line compares the cost of the best available split against the cost of simply testing all the primitives in a leaf. If splitting does not pay, stop.

Omitting it is the most common BVH bug. The builder keeps splitting until it reaches one primitive per leaf, producing a tree with far more nodes than necessary, higher memory traffic, and worse performance than a correctly terminated tree with leaves of two to four primitives.

The symptom is a node count several times larger than expected and traversal statistics showing many nodes visited but few primitives tested.

4 Traversal

Build quality sets the ceiling; traversal determines whether you reach it. Two details account for most of the difference between a fast traversal loop and a slow one.

- 1 **Shrink $t_{\max}$ on every hit.** Once a primitive has been hit at distance t , nothing beyond t can matter. Writing that into the ray immediately culls every subsequent node whose entry distance exceeds it.
- 2 **Visit the nearer child first.** Compare the two children's box entry distances and descend into the closer one, pushing the farther onto the stack. A hit found in the near subtree often culls the far one entirely. **This is worth roughly 2×** and costs one comparison.

Beyond those: precompute the reciprocal of the ray direction once rather than dividing in every slab test; keep the stack small and on the actual stack; and check the stored entry distance when popping a node, since a closer hit may have been found since it was pushed.

Statistic	Healthy range	What a bad value indicates
Nodes visited per ray	50–200	High: poor splits, or heavy bound overlap. Check the SAH implementation.
Primitives tested per ray	10–50	High: leaves are too large, or the termination test is too eager.

Statistic	Healthy range	What a bad value indicates
Average leaf size	1–4 primitives	Large: termination too aggressive. All ones: termination test missing.
Tree depth	about $2 \log_2 n$	Much deeper: degenerate splits — often all primitives sharing a centroid.
Build time	Seconds for 10^6 primitives	Minutes: you are not binning.

Instrument these now and keep them. Every performance question for the rest of the course is answered by looking at them first — a renderer that is slow because of traversal and one that is slow because of variance need completely different responses, and these numbers distinguish the two immediately.

5 Refinements

Technique	Idea	Gain
Spatial splits (SBVH)	Allow a primitive to be split across two nodes, as a kd-tree does, when the SAH says it pays. Recovers most of the kd-tree's advantage while keeping the BVH's other properties.	Up to ~30% on scenes with long thin triangles or widely varying primitive sizes. Memory is no longer strictly bounded.
Wide BVHs	Four or eight children per node, with all child boxes tested simultaneously using SIMD.	Shallower trees, fewer stack operations, better use of vector units. Standard in Embree.
Compressed nodes	Quantise child bounds to 8-bit offsets relative to the parent's box.	Several times less memory. At scale, memory bandwidth is the binding constraint, so this is often a speed win too.
Two-level BVHs	A top-level BVH over instance transforms, each instance referencing a shared bottom-level BVH over its geometry.	Essential for instancing — a forest of 10,000 identical trees stores one tree's BVH. This is exactly the BLAS/TLAS structure the GPU ray tracing APIs expose.
Ray reordering	Buffer up incoherent rays, sort them by origin and direction, then trace them in sorted order.	Recovers cache locality for incoherent rays — see below.

COHERENCE, NOT ARITHMETIC, IS THE REAL VARIABLE

Primary rays from a pinhole camera are highly **coherent**: neighbouring pixels generate nearly parallel rays that descend through almost the same sequence of nodes. Those nodes are already in cache, so traversal is effectively free in memory terms.

After a single diffuse bounce the directions are sampled at random. Two rays from adjacent pixels now travel to completely unrelated parts of the tree and touch disjoint memory.

The same BVH can be several times slower for incoherent rays than for coherent ones, and the cause is cache behaviour, not instruction count. This is why bounce 2 sometimes costs more than bounce 1 despite having fewer active paths.

It is also the reason GPU renderers use a **wavefront** architecture: rather than following each path to completion, they process all paths one bounce at a time, sorting rays between bounces to restore coherence. The sorting costs real time and still pays.

WRITE ONE, THEN USE SOMEONE ELSE'S

You will implement a BVH in this course, and you should. Understanding where traversal time goes is necessary for reasoning about everything downstream — when a render is slow, the first question is always whether it is traversal or variance, and you cannot answer it from outside.

For production, use Embree on the CPU or the hardware BVH on the GPU. They will be several times faster than yours, and the gap comes from years of SIMD tuning, cache-conscious memory layouts, and in the GPU's case dedicated silicon that you cannot compete with in software.

This is not a reason to skip the implementation. It is the difference between using a tool and being able to diagnose it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
PBRT 4th ed. — Chapter 7, Primitives and Intersection Acceleration	BVH construction including binned SAH, with complete code. The primary reading for this module.
Wald, Boulos & Shirley — Ray Tracing Deformable Scenes with Dynamic BVHs (free)	Where much of the modern BVH consensus comes from, including refitting and the case against kd-trees.
Stich, Friedrich & Dietrich — Spatial Splits in BVHs (SBVH, free)	The spatial-split hybrid of §5. Short and clearly argued.
Embree documentation and source	A production BVH you can read. Compare your traversal loop against theirs after you have written one.

Resource	What it covers for this module
Ray Tracing Gems — chapters on BVH traversal and compression (free)	Practical implementation detail, especially for the GPU case.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a BVH with median splitting and one with binned SAH. Report nodes visited and primitives tested per ray for both on the same scene. The SAH should win by 2–3×.
- 2 Measure C_{trav} and C_{isect} for your own renderer rather than using the textbook values, then rebuild with the measured constants and report the difference.
- 3 Remove the SAH termination test and report the resulting node count, memory use, and render time. Then put it back.
- 4 Implement near-first ordered traversal and measure the speedup against arbitrary ordering.
- 5 Instrument the five statistics of §4 and produce a heat-map image of nodes visited per pixel. The structure of the BVH will be visible in it, which is both useful and worth looking at.
- 6 Measure the cost of primary rays against the cost of rays after one, two, and three diffuse bounces, with all other factors held constant. Explain the trend using §5.
- 7 Build a scene of 10,000 instances of the same mesh. Compare memory use and build time for a flat BVH against a two-level one.
- 8 Benchmark your BVH against Embree on the same scene. Report the ratio honestly and identify where the difference comes from.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is brute-force intersection infeasible? Give the order of magnitude.
- 2 Give three differences between spatial and object subdivision.
- 3 Give four reasons the BVH became standard despite the kd-tree often being faster for static scenes.
- 4 State the geometric probability result the SAH rests on, and say why it involves surface area rather than volume.
- 5 What does the SAH minimise, and why will it deliberately build an unbalanced tree?
- 6 What does the SAH termination test do, and what happens without it?
- 7 Name the two traversal details that account for most of the performance, and say why each works.

- 8 What is ray coherence, why does it collapse after a diffuse bounce, and what do wavefront renderers do about it?